**FILMORA**

**DATABASE ARCHITECTURE & DESIGN SPECIFICATION**

**Project:** FILMORA – Free Online Movie Streaming Platform
**Document:** Database Architecture and Database Requirements
**Version:** 1.0
**Status:** Proposed Design
**Database Engine:** PostgreSQL (proposed)
**Backend:** Node.js + Express (proposed)
**Frontend:** HTML, CSS and JavaScript

---

**TABLE OF CONTENTS**


1. Project Overview
2. Database Architecture
3. User Management
4. Authentication and Security
5. User Profiles and Roles
6. Movie Management
7. Movie Genres and Categories
8. Actors, Directors and Cast
9. Movie Images and Media
10. Video Streaming Management
11. Subtitles and Audio Tracks
12. Watch History and Progress
13. Watchlists and Favorites
14. Reviews and Ratings
15. Notifications
16. Administration Management
17. Reports and Moderation
18. Website Configuration
19. Database Relationships
20. Database Security

21. Backup and Recovery
22. Final Database Inventory

---

**1. PROJECT OVERVIEW**

FILMORA is a free online movie streaming platform that allows registered users to discover movies and watch authorized full-length movies online.

The platform will provide:

- User registration and login.
- Movie browsing and discovery.
- Full-length movie streaming.
- Movie search and filtering.
- Movie categories and genres.
- User profiles.
- Personal watchlists.
- Viewing history.
- Movie reviews and ratings.
- Administrative movie management.
- Secure video delivery.

**1.1 Database objectives**

The database must:

- Store information reliably.
- Maintain relationships between records.
- Prevent unnecessary data duplication.
- Protect user information.
- Support movie streaming.
- Allow future platform expansion.
- Support administrative operations.

**2. DATABASE ARCHITECTURE**

FILMORA will use one primary relational database.

**Main database name:** filmora_db

Proposed supporting technologies:

| Component | Technology |
|---|---|
| Main database | PostgreSQL |
| Backend | Node.js + Express |
| Frontend | HTML, CSS, JavaScript |
| Movie storage | Object storage |
| Image storage | Object storage |
| Video delivery | CDN |
| Optional caching | Redis |

Movie files will not be stored directly inside PostgreSQL. The database will store file metadata and secure storage references.

**3. USER MANAGEMENT**

**Table 3.1: users**

Stores all registered FILMORA users.

| Column | Data type | Key |
|---|---|---|
| user_id | UUID | PK |
| username | VARCHAR(50) | UNIQUE |
| email | VARCHAR(255) | UNIQUE |
| password_hash | TEXT | |
| account_status | VARCHAR(20) | |
| email_verified | BOOLEAN | |
| created_at | TIMESTAMPTZ | |
| updated_at | TIMESTAMPTZ | |
| last_login_at | TIMESTAMPTZ | |

**Table 3.2: user_profiles**

Stores additional user information.

| Column | Data type | Key |
|---|---|---|
| profile_id | UUID | PK |
| user_id | UUID | FK |
| full_name | VARCHAR(150) | |
| profile_picture_id | UUID | FK |
| biography | TEXT | |
| country | VARCHAR(100) | |
| date_of_birth | DATE | |
| created_at | TIMESTAMPTZ | |
| updated_at | TIMESTAMPTZ | |

Each user has one profile.

**Table 3.3: user_preferences**

Stores individual viewing preferences.

| Column | Data type | Key |
|---|---|---|
| preference_id | UUID | PK |
| user_id | UUID | FK |
| preferred_language | VARCHAR(50) | |
| preferred_subtitle_language | VARCHAR(50) | |
| autoplay_enabled | BOOLEAN | |
| email_notifications | BOOLEAN | |
| created_at | TIMESTAMPTZ | |

**Table 3.4: user_devices**

Records devices associated with user accounts.

| Column | Data type | Key |
|---|---|---|
| device_id | UUID | PK |

| Column | Data type | Key |
|---|---|---|
| user_id | UUID | FK |
| device_name | VARCHAR(100) | |
| device_type | VARCHAR(50) | |
| last_seen_at | TIMESTAMPTZ | |
| created_at | TIMESTAMPTZ | |

## 4. AUTHENTICATION AND SECURITY

**Table 4.1: user_sessions**

Manages authenticated sessions.

| Column | Data type | Key |
|---|---|---|
| session_id | UUID | PK |
| user_id | UUID | FK |
| session_token_hash | TEXT | |
| ip_address | INET | |
| user_agent | TEXT | |
| expires_at | TIMESTAMPTZ | |
| revoked_at | TIMESTAMPTZ | |
| created_at | TIMESTAMPTZ | |

**Table 4.2: password_resets**

Manages password reset requests.

| Column | Data type | Key |
|---|---|---|
| reset_id | UUID | PK |
| user_id | UUID | FK |
| token_hash | TEXT | |
| expires_at | TIMESTAMPTZ | |

| Column | Data type | Key |
|---|---|---|
| used_at | TIMESTAMPTZ | |
| created_at | TIMESTAMPTZ | |

**Table 4.3: email_verifications**

Manages account email verification.

| Column | Data type | Key |
|---|---|---|
| verification_id | UUID | PK |
| user_id | UUID | FK |
| token_hash | TEXT | |
| expires_at | TIMESTAMPTZ | |
| verified_at | TIMESTAMPTZ | |
| created_at | TIMESTAMPTZ | |

## 5. USER ROLES AND PERMISSIONS

**Table 5.1: roles**

Defines available account roles.

| Column | Data type | Key |
|---|---|---|
| role_id | UUID | PK |
| role_name | VARCHAR(50) | UNIQUE |
| description | TEXT | |
| created_at | TIMESTAMPTZ | |

Initial roles:

- Administrator
- Content Manager
- Moderator
- Registered User

**Table 5.2: user_role_assignments**

Connects users with their roles.

| Column | Data type | Key |
|---|---|---|
| assignment_id | UUID | PK |
| user_id | UUID | FK |
| role_id | UUID | FK |
| assigned_at | TIMESTAMPTZ | |
| assigned_by | UUID | FK |

**Table 5.3: permissions**

Stores available system permissions.

| Column | Data type | Key |
|---|---|---|
| permission_id | UUID | PK |
| permission_name | VARCHAR(100) | UNIQUE |
| description | TEXT | |

**Table 5.4: role_permissions**

Connects roles to permissions.

| Column | Data type | Key |
|---|---|---|
| role_permission_id | UUID | PK |
| role_id | UUID | FK |
| permission_id | UUID | FK |

**6. MOVIE MANAGEMENT**

**Table 6.1: movies**

The main movie information table.

| Column | Data type | Key |
|---|---|---|
| movie_id | UUID | PK |
| title | VARCHAR(255) | |
| slug | VARCHAR(280) | UNIQUE |
| description | TEXT | |

| Column | Data type | Key |
|---|---|---|
| release_date | DATE | |
| release_year | SMALLINT | |
| duration_minutes | INTEGER | |
| original_language | VARCHAR(50) | |
| age_rating | VARCHAR(20) | |
| country_of_origin | VARCHAR(100) | |
| content_status | VARCHAR(30) | |
| featured | BOOLEAN | |
| published_at | TIMESTAMPTZ | |
| created_at | TIMESTAMPTZ | |
| updated_at | TIMESTAMPTZ | |

Possible content statuses:

- Draft
- Processing
- Published
- Unpublished
- Archived

**Table 6.2: movie_languages**

Stores languages associated with each movie.

| Column | Data type | Key |
|---|---|---|
| movie_language_id | UUID | PK |
| movie_id | UUID | FK |
| language_code | VARCHAR(10) | |
| language_name | VARCHAR(100) | |
| language_type | VARCHAR(30) | |

## 7. MOVIE GENRES AND CATEGORIES

**Table 7.1: genres**

| Column | Data type | Key |
|---|---|---|
| genre_id | UUID | PK |
| genre_name | VARCHAR(100) | UNIQUE |
| description | TEXT | |
| created_at | TIMESTAMPTZ | |

Examples:

- Action
- Adventure
- Comedy
- Drama
- Horror
- Romance
- Science Fiction
- Thriller
- Animation
- Documentary

**Table 7.2: movie_genres**

Connects movies to genres.

| Column | Data type | Key |
|---|---|---|
| movie_genre_id | UUID | PK |
| movie_id | UUID | FK |
| genre_id | UUID | FK |

A movie can have multiple genres.

**Table 7.3: categories**

Defines website browsing collections.

| Column | Data type | Key |
|---|---|---|
| category_id | UUID | PK |
| category_name | VARCHAR(100) | UNIQUE |
| description | TEXT | |
| display_order | INTEGER | |
| is_active | BOOLEAN | |

Examples:

- Trending Movies
- Recently Added
- Most Watched
- Featured Movies
- Top Rated

**Table 7.4: category_movies**

Connects movies to website categories.

| Column | Data type | Key |
|---|---|---|
| category_movie_id | UUID | PK |
| category_id | UUID | FK |
| movie_id | UUID | FK |
| display_order | INTEGER | |

**8. ACTORS, DIRECTORS AND CAST**

**Table 8.1: people**

Stores information about actors, directors and other film professionals.

| Column | Data type | Key |
|---|---|---|
| person_id | UUID | PK |
| full_name | VARCHAR(200) | |
| biography | TEXT | |

| Column | Data type | Key |
|---|---|---|
| birth_date | DATE | |
| profile_image_id | UUID | FK |
| created_at | TIMESTAMPTZ | |

**Table 8.2: movie_credits**

Connects people to movies and defines their roles.

| Column | Data type | Key |
|---|---|---|
| credit_id | UUID | PK |
| movie_id | UUID | FK |
| person_id | UUID | FK |
| credit_type | VARCHAR(30) | |
| character_name | VARCHAR(150) | |
| display_order | INTEGER | |

Examples of credit types:

- Actor
- Director
- Producer
- Writer
- Cinematographer

## 9. MOVIE IMAGES AND MEDIA

**Table 9.1: media_assets**

Stores information about uploaded media files.

| Column | Data type | Key |
|---|---|---|
| media_asset_id | UUID | PK |
| file_name | VARCHAR(255) | |
| storage_key | TEXT | |

| Column | Data type | Key |
|---|---|---|
| storage_provider | VARCHAR(50) | |
| mime_type | VARCHAR(100) | |
| file_size_bytes | BIGINT | |
| width | INTEGER | |
| height | INTEGER | |
| uploaded_by | UUID | FK |
| created_at | TIMESTAMPTZ | |

**Table 9.2: movie_media**

Associates images with movies.

| Column | Data type | Key |
|---|---|---|
| movie_media_id | UUID | PK |
| movie_id | UUID | FK |
| media_asset_id | UUID | FK |
| media_type | VARCHAR(30) | |
| is_primary | BOOLEAN | |
| created_at | TIMESTAMPTZ | |

Media types:

- Poster
- Backdrop
- Thumbnail
- Promotional Image

**10. VIDEO STREAMING MANAGEMENT**

**Table 10.1: video_assets**

Stores the main movie video asset information.

| Column | Data type | Key |
|---|---|---|

| Column | Data type | Key |
|---|---|---|
| video_asset_id | UUID | PK |
| movie_id | UUID | FK |
| storage_key | TEXT | |
| storage_provider | VARCHAR(50) | |
| container_format | VARCHAR(20) | |
| duration_seconds | INTEGER | |
| processing_status | VARCHAR(30) | |
| created_at | TIMESTAMPTZ | |

**Table 10.2: video_versions**

Stores different encoded streaming versions.

| Column | Data type | Key |
|---|---|---|
| video_version_id | UUID | PK |
| video_asset_id | UUID | FK |
| resolution | VARCHAR(20) | |
| bitrate_kbps | INTEGER | |
| codec | VARCHAR(50) | |
| manifest_storage_key | TEXT | |
| status | VARCHAR(30) | |
| created_at | TIMESTAMPTZ | |

Example resolutions:

- 360p
- 480p
- 720p
- 1080p

**Table 10.3: streaming_sessions**

Tracks active movie viewing sessions.

| Column | Data type | Key |
|---|---|---|
| streaming_session_id | UUID | PK |
| user_id | UUID | FK |
| movie_id | UUID | FK |
| device_id | UUID | FK |
| started_at | TIMESTAMPTZ | |
| ended_at | TIMESTAMPTZ | |
| session_status | VARCHAR(30) | |

## 11. SUBTITLES AND AUDIO TRACKS

**Table 11.1: subtitle_tracks**

| Column | Data type | Key |
|---|---|---|
| subtitle_id | UUID | PK |
| video_asset_id | UUID | FK |
| language_code | VARCHAR(10) | |
| language_name | VARCHAR(100) | |
| storage_key | TEXT | |
| format | VARCHAR(20) | |
| is_default | BOOLEAN | |

**Table 11.2: audio_tracks**

| Column | Data type | Key |
|---|---|---|
| audio_track_id | UUID | PK |
| video_asset_id | UUID | FK |
| language_code | VARCHAR(10) | |
| language_name | VARCHAR(100) | |

| Column | Data type | Key |
|---|---|---|
| codec | VARCHAR(50) | |
| is_default | BOOLEAN | |

## 12. WATCH HISTORY AND PROGRESS

### Table 12.1: watch_progress

Stores the latest viewing position for each user and movie.

| Column | Data type | Key |
|---|---|---|
| progress_id | UUID | PK |
| user_id | UUID | FK |
| movie_id | UUID | FK |
| position_seconds | INTEGER | |
| completed | BOOLEAN | |
| last_watched_at | TIMESTAMPTZ | |
| updated_at | TIMESTAMPTZ | |

A unique constraint should exist on user_id and movie_id.

### Table 12.2: watch_history

Stores individual viewing events.

| Column | Data type | Key |
|---|---|---|
| history_id | UUID | PK |
| user_id | UUID | FK |
| movie_id | UUID | FK |
| started_at | TIMESTAMPTZ | |
| finished_at | TIMESTAMPTZ | |
| watched_seconds | INTEGER | |
| completed | BOOLEAN | |

## 13. WATCHLISTS AND FAVORITES

**Table 13.1: watchlists**

| Column | Data type | Key |
|---|---|---|
| watchlist_id | UUID | PK |
| user_id | UUID | FK |
| list_name | VARCHAR(100) | |
| is_public | BOOLEAN | |
| created_at | TIMESTAMPTZ | |

**Table 13.2: watchlist_items**

| Column | Data type | Key |
|---|---|---|
| watchlist_item_id | UUID | PK |
| watchlist_id | UUID | FK |
| movie_id | UUID | FK |
| added_at | TIMESTAMPTZ | |

**Table 13.3: favorites**

| Column | Data type | Key |
|---|---|---|
| favorite_id | UUID | PK |
| user_id | UUID | FK |
| movie_id | UUID | FK |
| created_at | TIMESTAMPTZ | |

**14. REVIEWS AND RATINGS**

**Table 14.1: reviews**

| Column | Data type | Key |
|---|---|---|
| review_id | UUID | PK |
| user_id | UUID | FK |
| movie_id | UUID | FK |

| Column | Data type | Key |
|---|---|---|
| rating | SMALLINT | |
| review_text | TEXT | |
| review_status | VARCHAR(20) | |
| created_at | TIMESTAMPTZ | |
| updated_at | TIMESTAMPTZ | |

Rating must be between 1 and 5.

A user should only be allowed one active review per movie.

**15. NOTIFICATIONS**

**Table 15.1: notifications**

| Column | Data type | Key |
|---|---|---|
| notification_id | UUID | PK |
| user_id | UUID | FK |
| title | VARCHAR(200) | |
| message | TEXT | |
| notification_type | VARCHAR(50) | |
| is_read | BOOLEAN | |
| created_at | TIMESTAMPTZ | |

**16. ADMINISTRATION MANAGEMENT**

**Table 16.1: admin_logs**

Records administrative actions.

| Column | Data type | Key |
|---|---|---|
| log_id | UUID | PK |
| admin_user_id | UUID | FK |
| action | VARCHAR(100) | |
| entity_type | VARCHAR(50) | |

| Column | Data type | Key |
|---|---|---|
| entity_id | UUID | |
| description | TEXT | |
| created_at | TIMESTAMPTZ | |

**Table 16.2: movie_uploads**

Tracks movie upload and processing jobs.

| Column | Data type | Key |
|---|---|---|
| upload_id | UUID | PK |
| movie_id | UUID | FK |
| uploaded_by | UUID | FK |
| storage_key | TEXT | |
| upload_status | VARCHAR(30) | |
| processing_error | TEXT | |
| created_at | TIMESTAMPTZ | |
| completed_at | TIMESTAMPTZ | |

**Table 16.3: content_rights**

Records permissions and licensing for movie content.

| Column | Data type | Key |
|---|---|---|
| rights_id | UUID | PK |
| movie_id | UUID | FK |
| rights_holder | VARCHAR(255) | |
| license_reference | TEXT | |
| territory | VARCHAR(100) | |
| valid_from | DATE | |
| valid_until | DATE | |

| Column | Data type | Key |
|---|---|---|
| rights_status | VARCHAR(30) | |
| created_at | TIMESTAMPTZ | |

## 17. REPORTS AND MODERATION

### Table 17.1: content_reports

Stores reports submitted by users.

| Column | Data type | Key |
|---|---|---|
| report_id | UUID | PK |
| reporter_user_id | UUID | FK |
| target_type | VARCHAR(30) | |
| target_id | UUID | |
| reason | VARCHAR(100) | |
| description | TEXT | |
| report_status | VARCHAR(30) | |
| reviewed_by | UUID | FK |
| created_at | TIMESTAMPTZ | |
| resolved_at | TIMESTAMPTZ | |

### Table 17.2: moderation_actions

Records moderation decisions.

| Column | Data type | Key |
|---|---|---|
| moderation_id | UUID | PK |
| moderator_user_id | UUID | FK |
| report_id | UUID | FK |
| action_type | VARCHAR(50) | |
| reason | TEXT | |

| Column | Data type | Key |
|---|---|---|
| created_at | TIMESTAMPTZ | |

**18. WEBSITE CONFIGURATION**

**Table 18.1: site_settings**

Stores configurable website settings.

| Column | Data type | Key |
|---|---|---|
| setting_id | UUID | PK |
| setting_key | VARCHAR(150) | UNIQUE |
| setting_value | JSONB | |
| description | TEXT | |
| updated_at | TIMESTAMPTZ | |
| updated_by | UUID | FK |

**19. DATABASE RELATIONSHIPS**

The main relationships will be:

- One user has one profile.
- One user can have multiple sessions.
- One user can have multiple watchlists.
- One user can write multiple reviews.
- One movie can have multiple genres.
- One genre can contain multiple movies.
- One movie can have multiple actors and directors.
- One movie can have multiple video assets.
- One video asset can have multiple video versions.
- One movie can have multiple subtitles and audio tracks.
- One movie can appear in multiple categories.
- One user can have multiple watch history records.
- One movie can have multiple favorites.

**20. DATABASE SECURITY**

FILMORA must implement:

1. Secure password hashing.
2. Unique email and username constraints.
3. Role-based access control.
4. Protected backend endpoints.
5. Parameterized SQL queries.
6. Secure session management.
7. Database connection restrictions.
8. Regular security updates.
9. Restricted access to video storage.
10. Audit logs for administrative actions.

The application must never expose database credentials or private storage credentials to the browser.

**21. BACKUP AND RECOVERY**

The database must have:

- Automated backups.
- Backup retention policies.
- Point-in-time recovery where supported.
- Regular restoration testing.
- Restricted backup access.
- Monitoring of database health.

Movie files and images must also have their own storage backup and recovery strategy.

**22. FINAL DATABASE INVENTORY**

The proposed architecture contains the following 33 tables:

| No. | Table | Department |
|---|---|---|
| 1 | users | User management |
| 2 | user_profiles | User management |

| No. | Table | Department |
|---|---|---|
| 3 | user_preferences | User management |
| 4 | user_devices | User management |
| 5 | user_sessions | Authentication |
| 6 | password_resets | Authentication |
| 7 | email_verifications | Authentication |
| 8 | roles | Permissions |
| 9 | user_role_assignments | Permissions |
| 10 | permissions | Permissions |
| 11 | role_permissions | Permissions |
| 12 | movies | Movies |
| 13 | movie_languages | Movies |
| 14 | genres | Genres |
| 15 | movie_genres | Genres |
| 16 | categories | Categories |
| 17 | category_movies | Categories |
| 18 | people | Cast |
| 19 | movie_credits | Cast |
| 20 | media_assets | Media |
| 21 | movie_media | Media |
| 22 | video_assets | Streaming |
| 23 | video_versions | Streaming |
| 24 | streaming_sessions | Streaming |
| 25 | subtitle_tracks | Streaming |
| 26 | audio_tracks | Streaming |

| No. | Table | Department |
|---|---|---|
| 27 | watch_progress | User activity |
| 28 | watch_history | User activity |
| 29 | watchlists | User activity |
| 30 | watchlist_items | User activity |
| 31 | favorites | User activity |
| 32 | reviews | User activity |
| 33 | notifications | User activity |

**Additional administrative tables:**

- admin_logs
- movie_uploads
- content_rights
- content_reports
- moderation_actions
- site_settings

**Total proposed tables: 39.**

---

**FINAL DESIGN PRINCIPLE**

FILMORA will use a centralized PostgreSQL database with logically separated tables and relationships.

Large media files will be managed through external storage services, while the database will manage their metadata, permissions and references.

This architecture is designed to support the initial website and future expansion.

**END OF DATABASE ARCHITECTURE SPECIFICATION**

*FILMORA — YOUR WORLD OF MOVIES.*